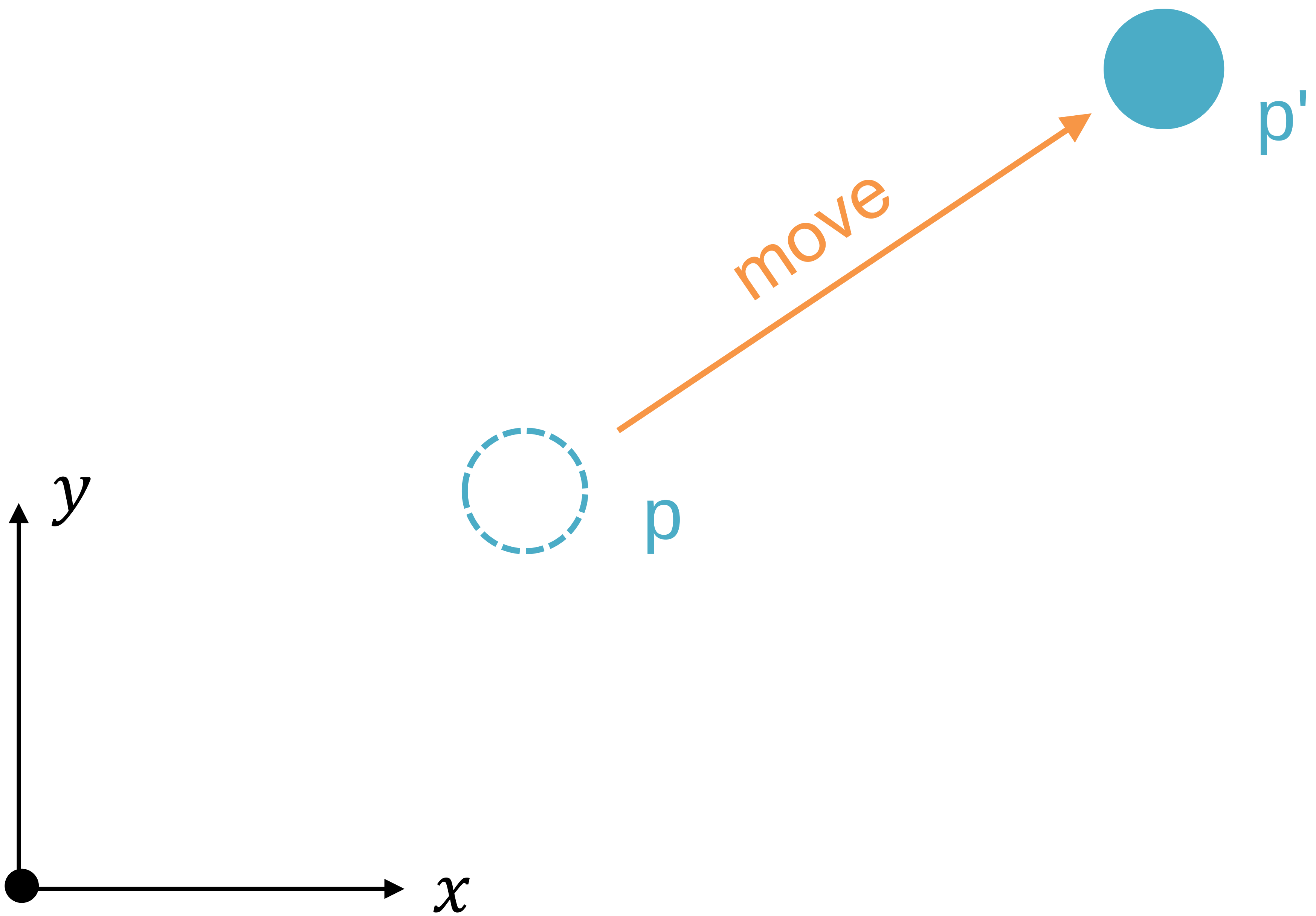
Class

Object-Oriented Programming with C++


move
p'
p
y
x

Point

```
typedef struct point {
    int x;
    int y;
} Point;
```

Point

```
typedef struct point {
    int x;
    int y;
} Point;

Point a;
a.x = 1;a.y = 2;
```

Point

```
typedef struct point {
    int x;
    int y;
} Point;

Point a;
a.x = 1;a.y = 2;
void print(const Point* p) {
    printf("%d %d\n", p->x, p->y);
}
```

Point

```
typedef struct point {
    int x;
    int y;
} Point;

Point a;
a.x = 1;a.y = 2;
void print(const Point* p) {
    printf("%d %d\n", p->x, p->y);
}
print(&a);
```

move (dx, dy)?

move (dx, dy)?

```
void move(Point* p, int dx, int dy) {
    p->x += dx;
    p->y += dy;
}
```

Prototypes

```
typedef struct point {
    int x;
    int y;
} Point;

void print(const Point* p);
void move(Point* p, int dx, int dy);
```

Usage

```
Point a;
Point b;
a.x = b.x = 1; a.y = b.y = 1;
move(&a,2,2);
print(&a);
print(&b);
```

C++ version

```
class Point {
public:
    void init(int x, int y);
    void move(int dx, int dy);
    void print() const;

private:
    int x;
    int y;
} ;
```

Implementations

```
void Point::init(int ix, int iy) {
    x = ix; y = iy;
}
void Point::move(int dx, int dy) {
    x+= dx; y+= dy;
}
void Point::print() const {
    cout << x << ' ' << y << endl;
}
```

C vs. C++

```c
typedef struct point {
    int x;
    int y;
} Point;

void print(const Point* p);
void move(Point* p,
          int dx,
          int dy);

Point a;
a.x = 1;
a.y = 2;
move(&a, 2, 2);
print(&a);
```

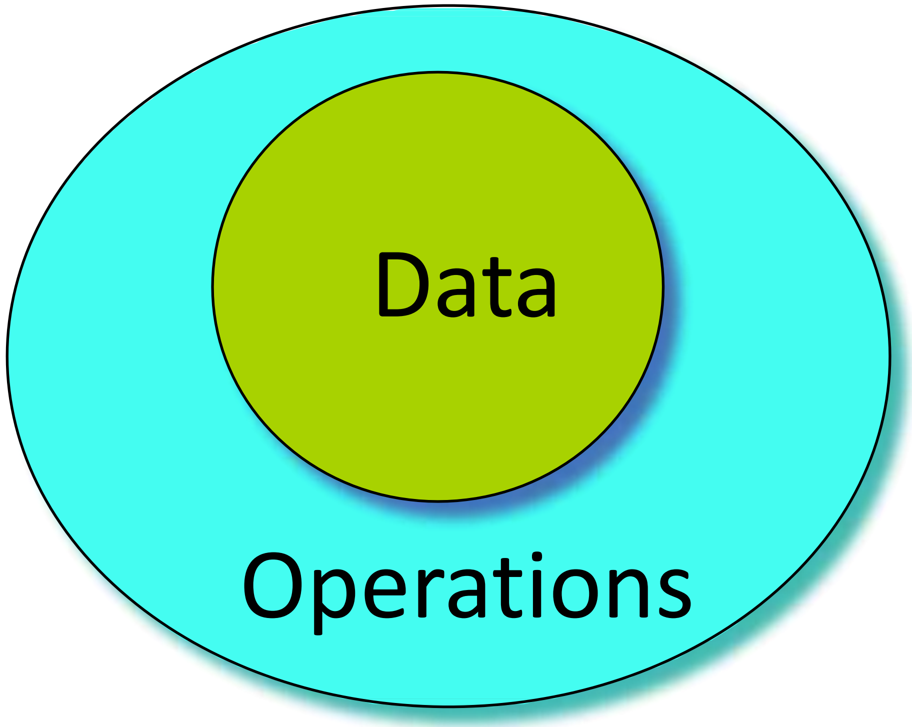
```cpp
class Point {
public:
    void init(int x, int y);
    void print() const;
    void move(int dx, int dy);

private:
    int x;
    int y;
};

Point a;
a.init(1, 2);
a.move(2, 2);
a.print();
```

Objects = Attributes + Services

- Data: the properties or status
- Operations: the functions



this: the hidden parameter

- **this** is a hidden parameter for all member functions, with the type of the struct
  ```
  void Stash::initialize(int sz)
  ```
  ➔ (can be regarded as)
  ```
  void Stash::initialize(Stash*this, int
   sz)
  ```
- To call the function, you must specify a variable
  ```
  Stash a;
  a.initialize(10);
  ```
  ➔ (can be regarded as)
  ```
  Stash::initialize(&a,10);
  ```

this: the pointer to the variable

- Inside member functions, you can use **this** as the pointer to the variable that calls the function.

- **this** is a natural local variable of all structs member functions that you can not define, but can use it directly.

Objects

- In C++, an object is just a variable, and the purest definition is “a region of storage”.
- The struct variables mentioned before are just objects in C++.

Ticket Machine

- Ticket machines print a ticket when a customer inserts the correct money for their fare.
- Our ticket machines work by customers' inserting money into them, and then requesting a ticket to be printed. A machine keeps a running total of the amount of money it has collected throughout its operation.

Procedure-Oriented

- Step to the machine
- Insert money into the machine
- The machine prints a ticket
- Take the ticket and leave

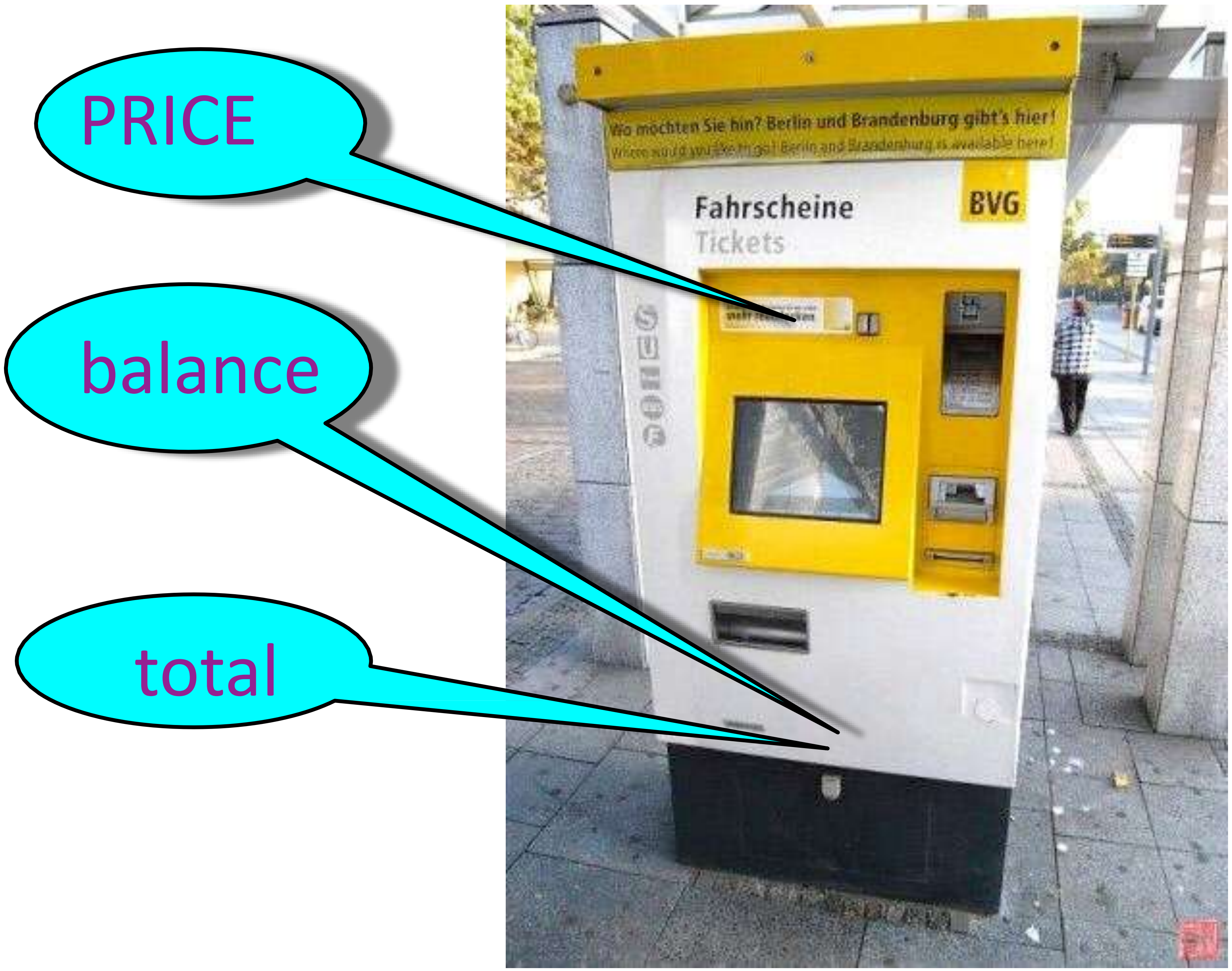
Procedure-Oriented

- Step to the machine
- Insert money into the machine

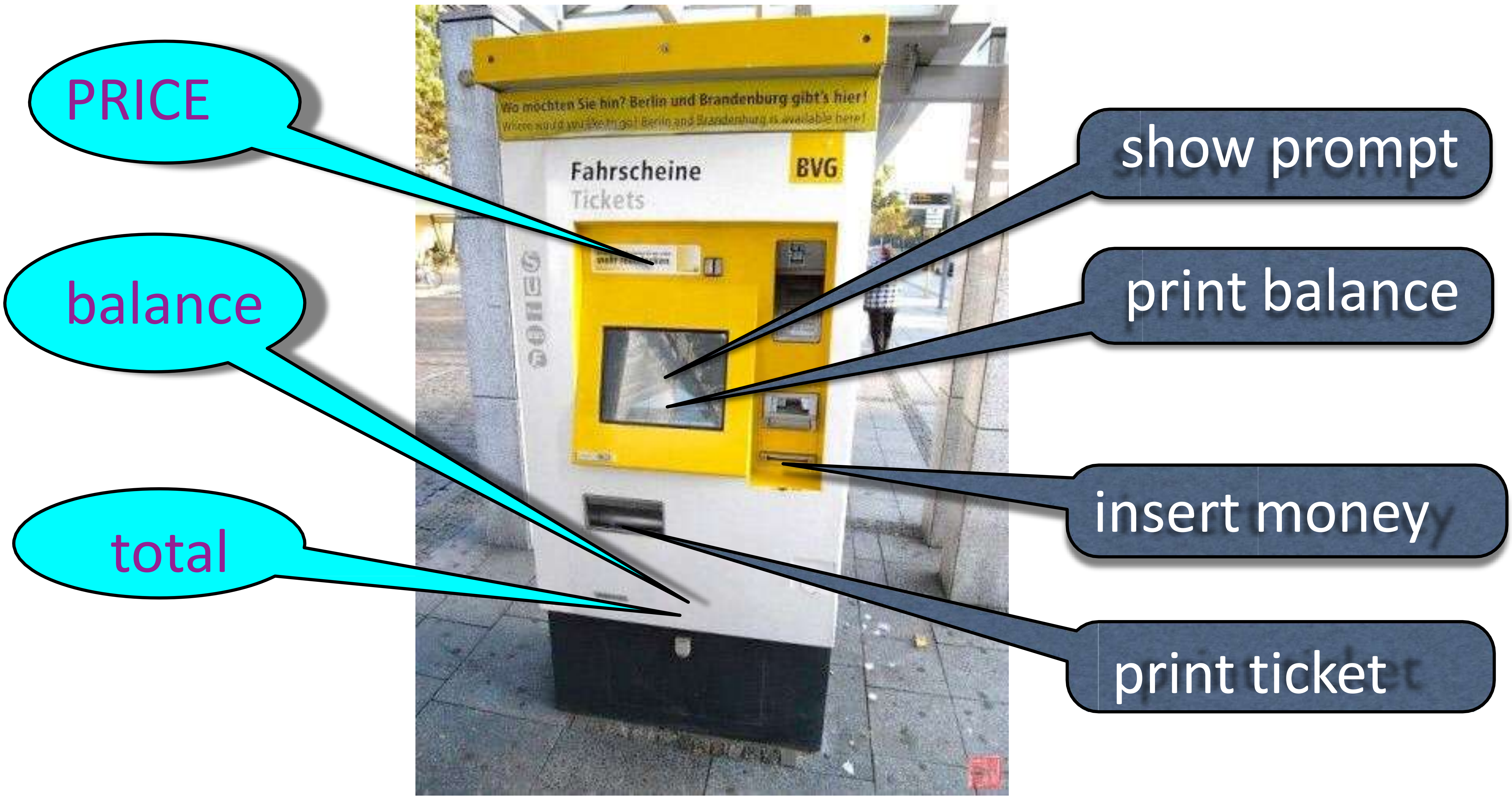
We make a program simulate the procedure of buying tickets. It works. But there is no such machine. There's nothing left for the further development.

Something is there

Something is there



Something is there

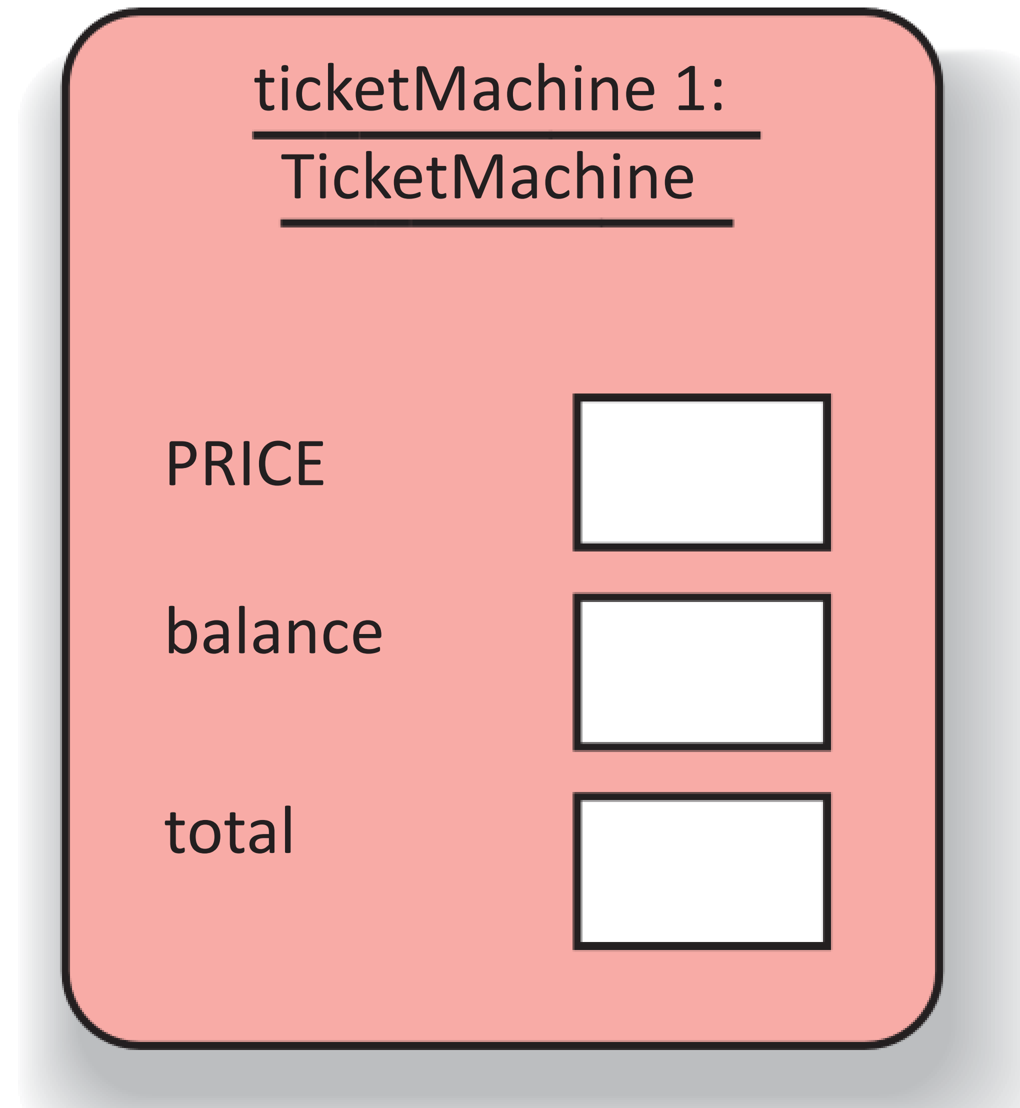


Something is there

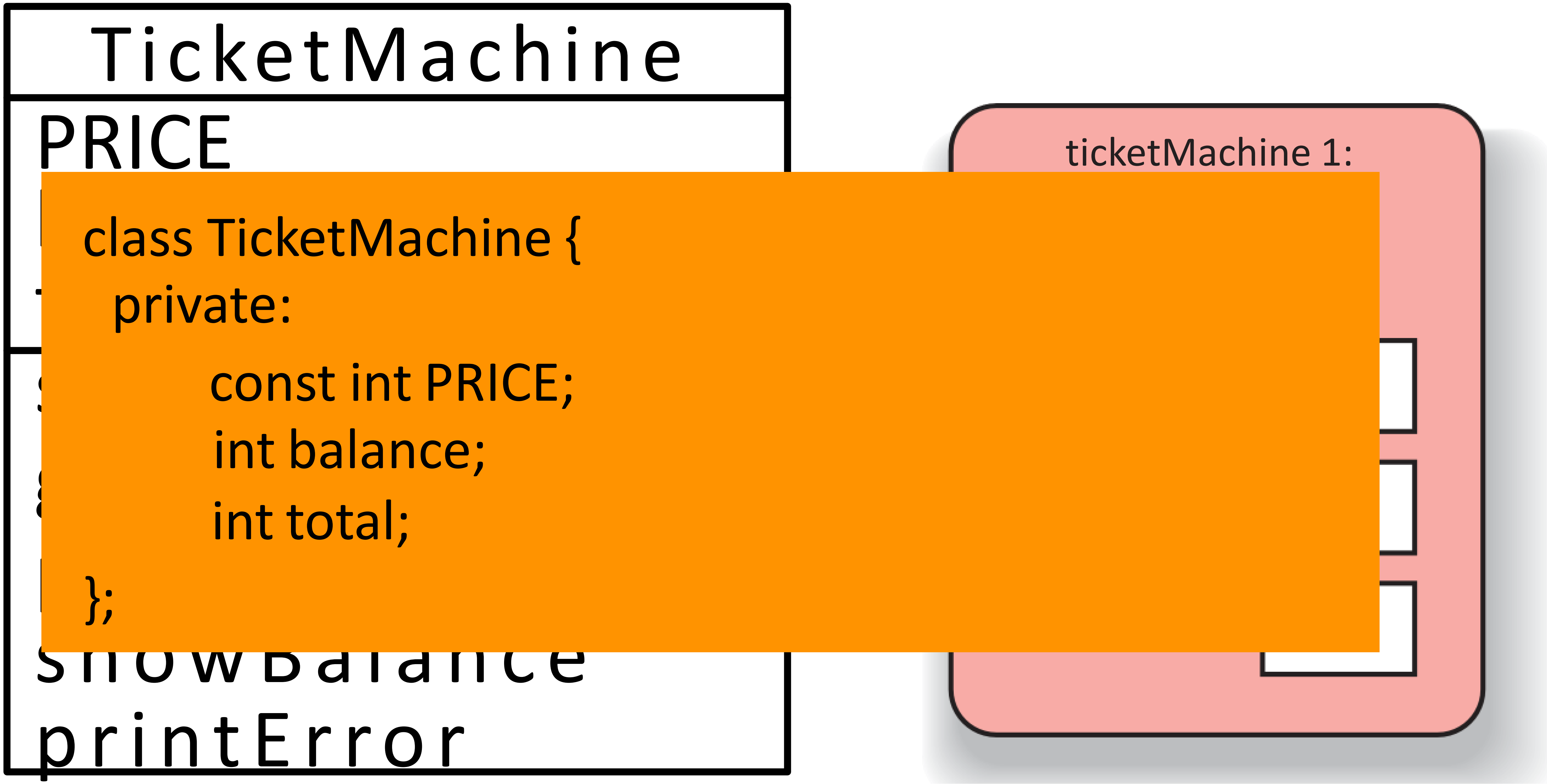
TicketMachine
PRICE balance total
showPrompt getMoney printTicket showBalance printError

Something is there

TicketMachine
PRICE balance total
showPrompt getMoney printTicket showBalance printError



Turn it into code



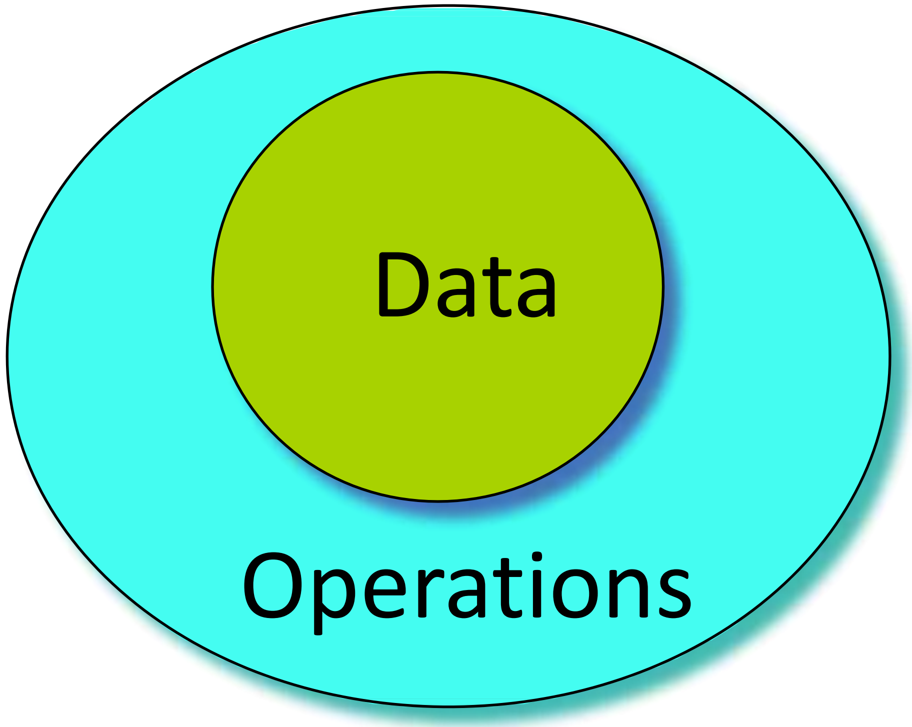

```
class TicketMachine {
  private:
        const int PRICE;
        int balance;
        int total;
};
```

Turn it into code

```
class TicketMachine {
public:
    void showPrompt();
    void getMoney();
    void printTicket();
    void showBalance();
    void printError();
private:
    const int PRICE;
    int balance;
    int total;
};
```

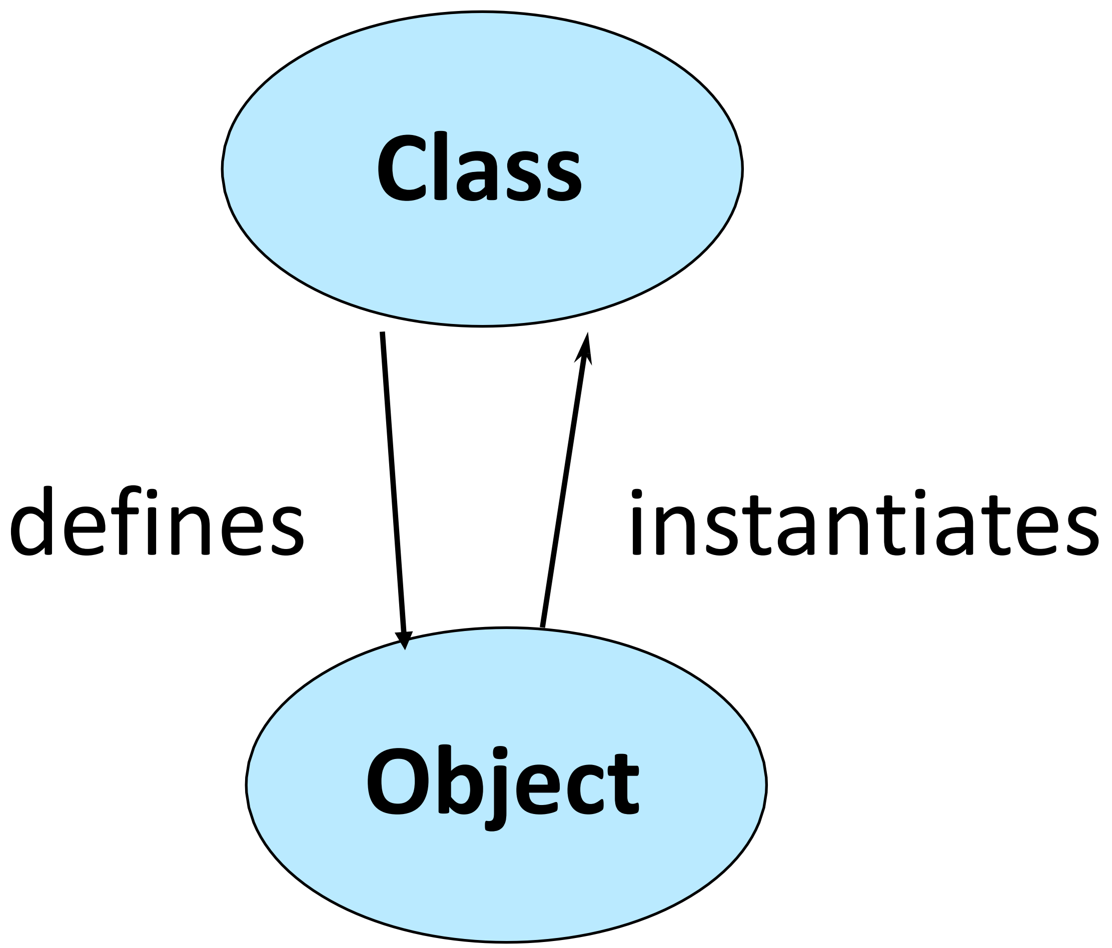
Objects = Attributes + Services

- Data: the properties or status
- Operations: the functions



Object vs. Class

- Objects (cat)
  - Represent things, events
  - Respond to messages at run-time
- Classes (cat class)
  - Define properties of instances
  - Act like types in C++



OOP Characteristics

1. Everything is an object.
2. A program is a bunch of objects telling each other what to do by sending messages.
3. Each object has its own memory made up of other objects.
4. Every object has a type.
5. All objects of a particular type can receive the same messages.

Definition of a Class

- In C++, separated .h and .cpp files are used to define one class.
- Class declaration and member function prototypes are in the header file (.h).
- All the bodies of these functions are in the source file (.cpp).
- PImpl technique: *debatable*, hides private members and removes compilation dependency.

:: resolver

- <Class Name>::<function name>
- ::<function name>

```
void S::f() {
   ::f(); // Would be recursive otherwise!
   ::a++; // Select the global 'a'
   a--;   // The 'a' at class scope
}
```

Compilation unit

- The *compiler* sees only one .cpp file, and generates one .obj file.
- The *linker* links all .obj files into one executable file.
- To provide information about functions in other .cpp files, use .h file.

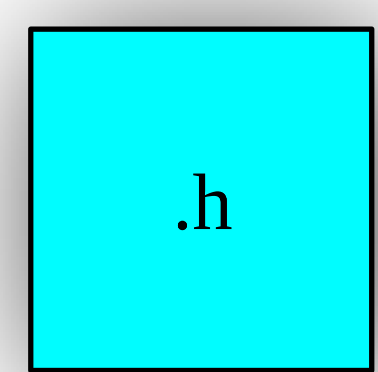
The header files

- If a function is declared in a header file, you *must* include the header file everywhere the function is used and where the function is defined.
- If a class is declared in a header file, you *must* include the header file everywhere the class is used and where class member functions are defined.

Header = interface

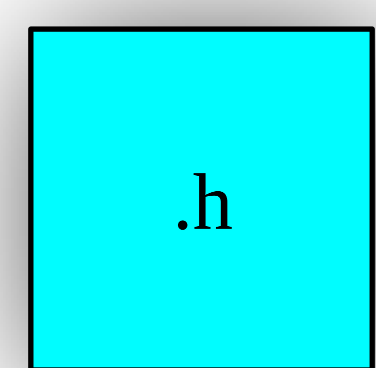
- The header is a contract between you and the user of your code.
- The compiler enforces the contract by requiring you to declare all structures and functions before they are used.

Structure of C++ program

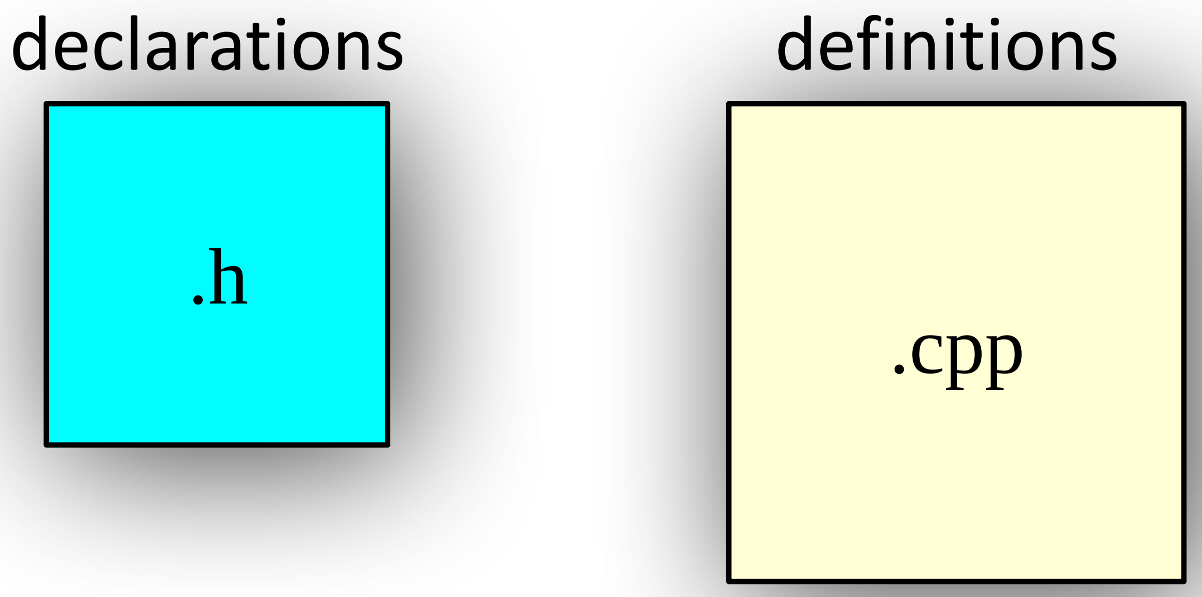


Structure of C++ program

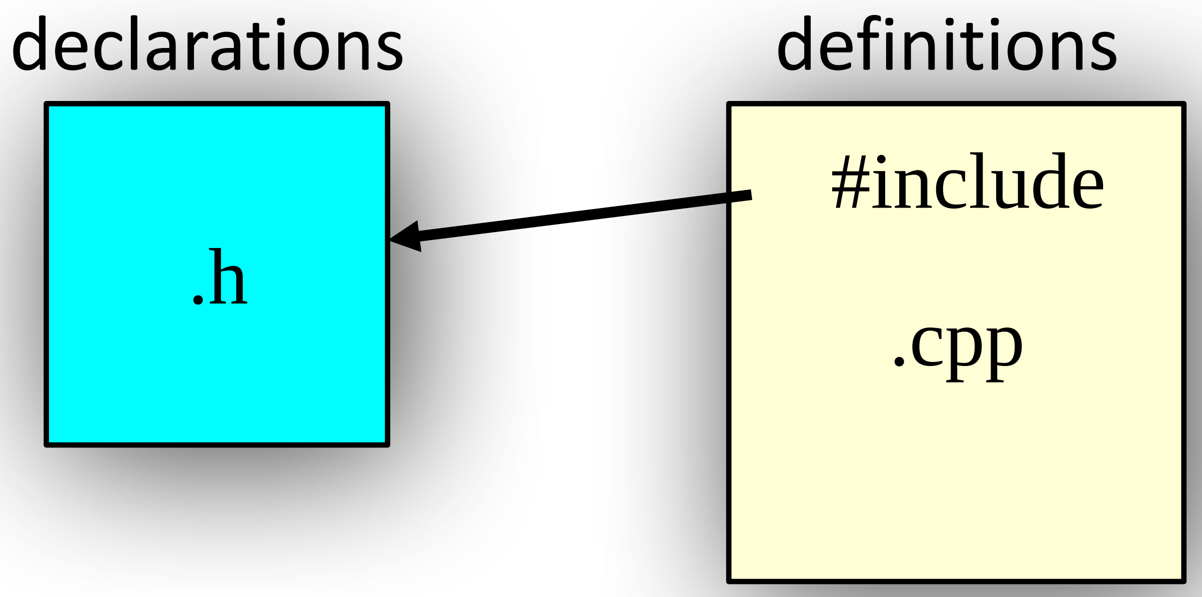
declarations



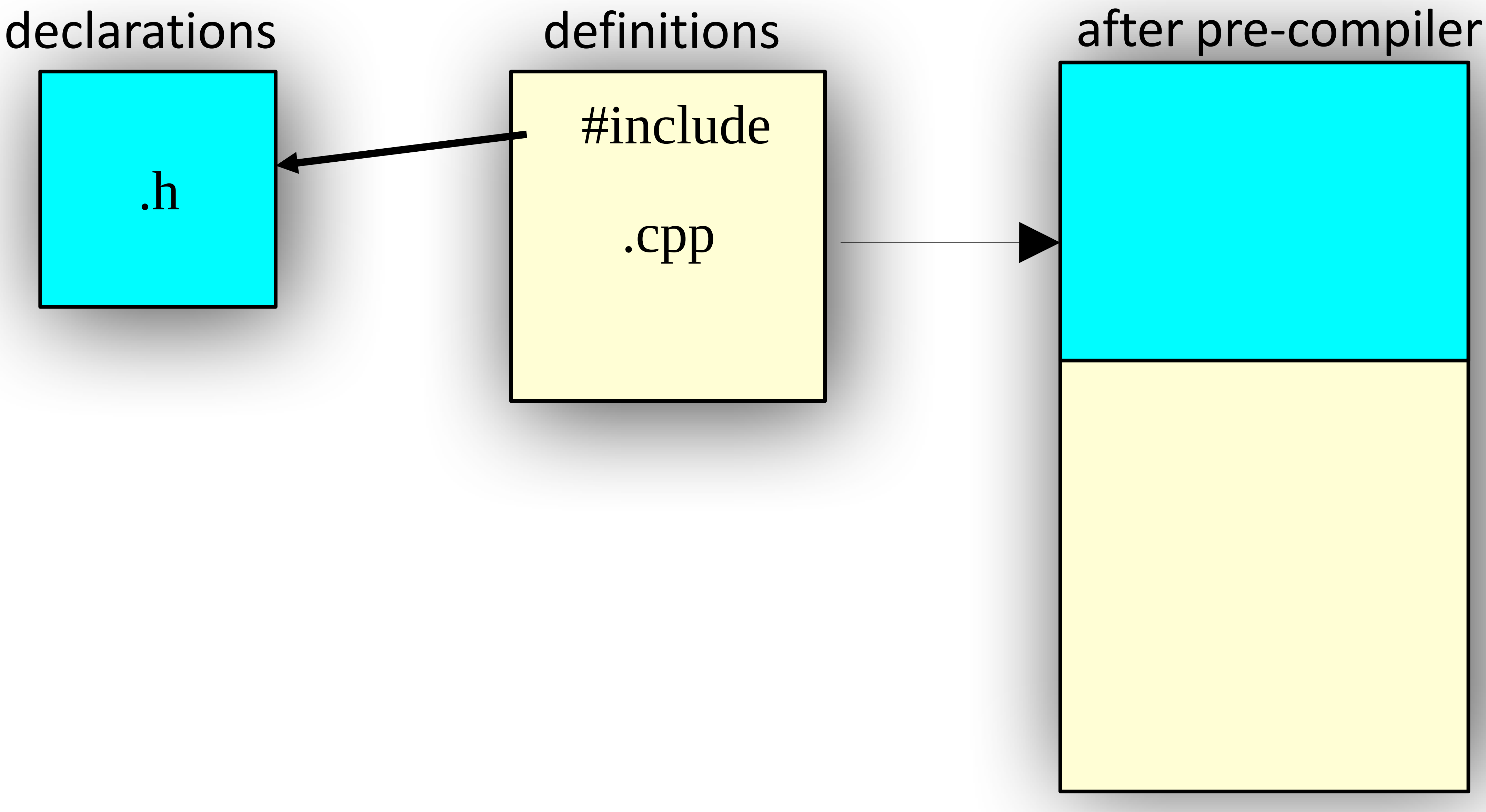
Structure of C++ program



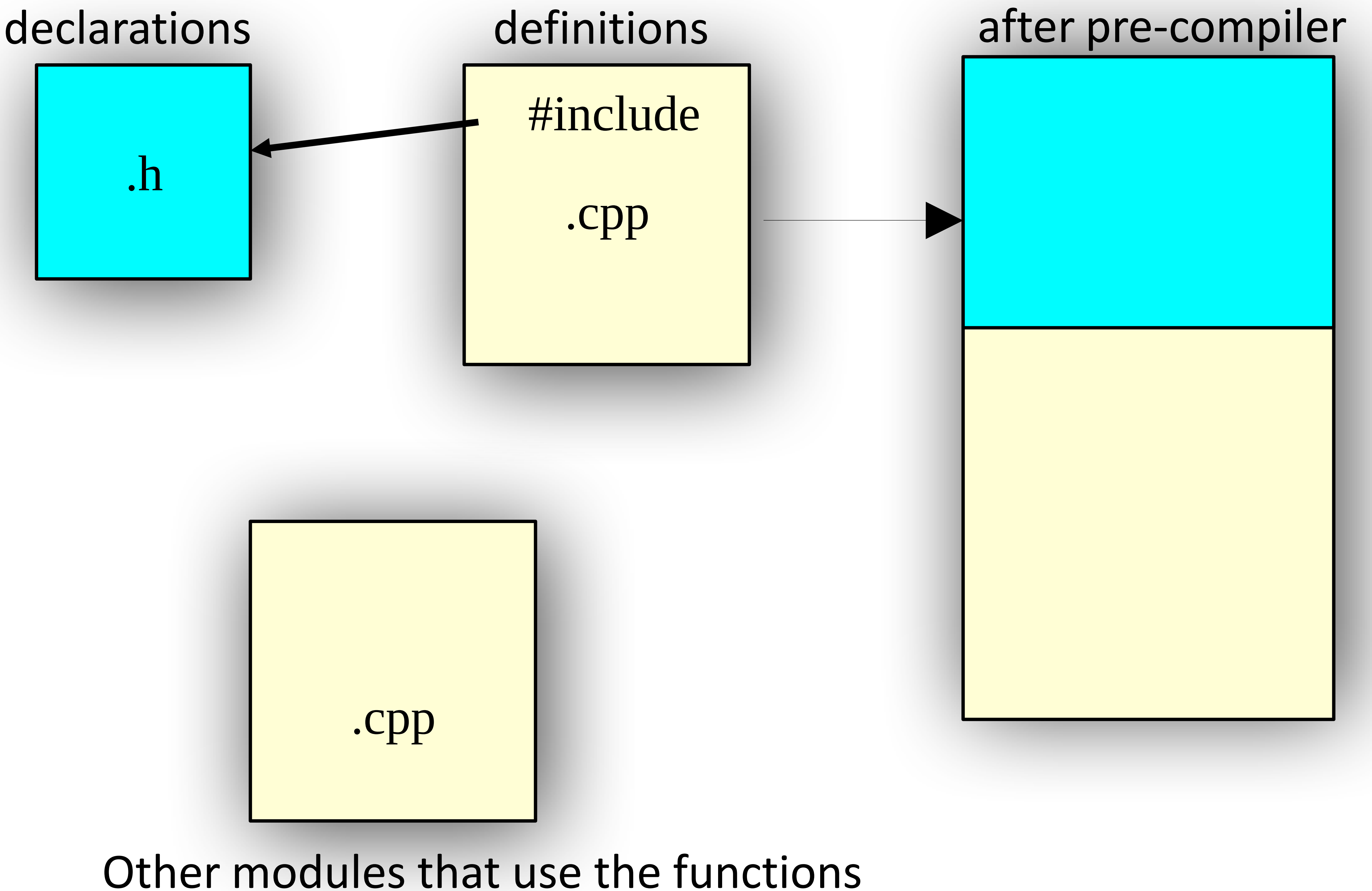
Structure of C++ program



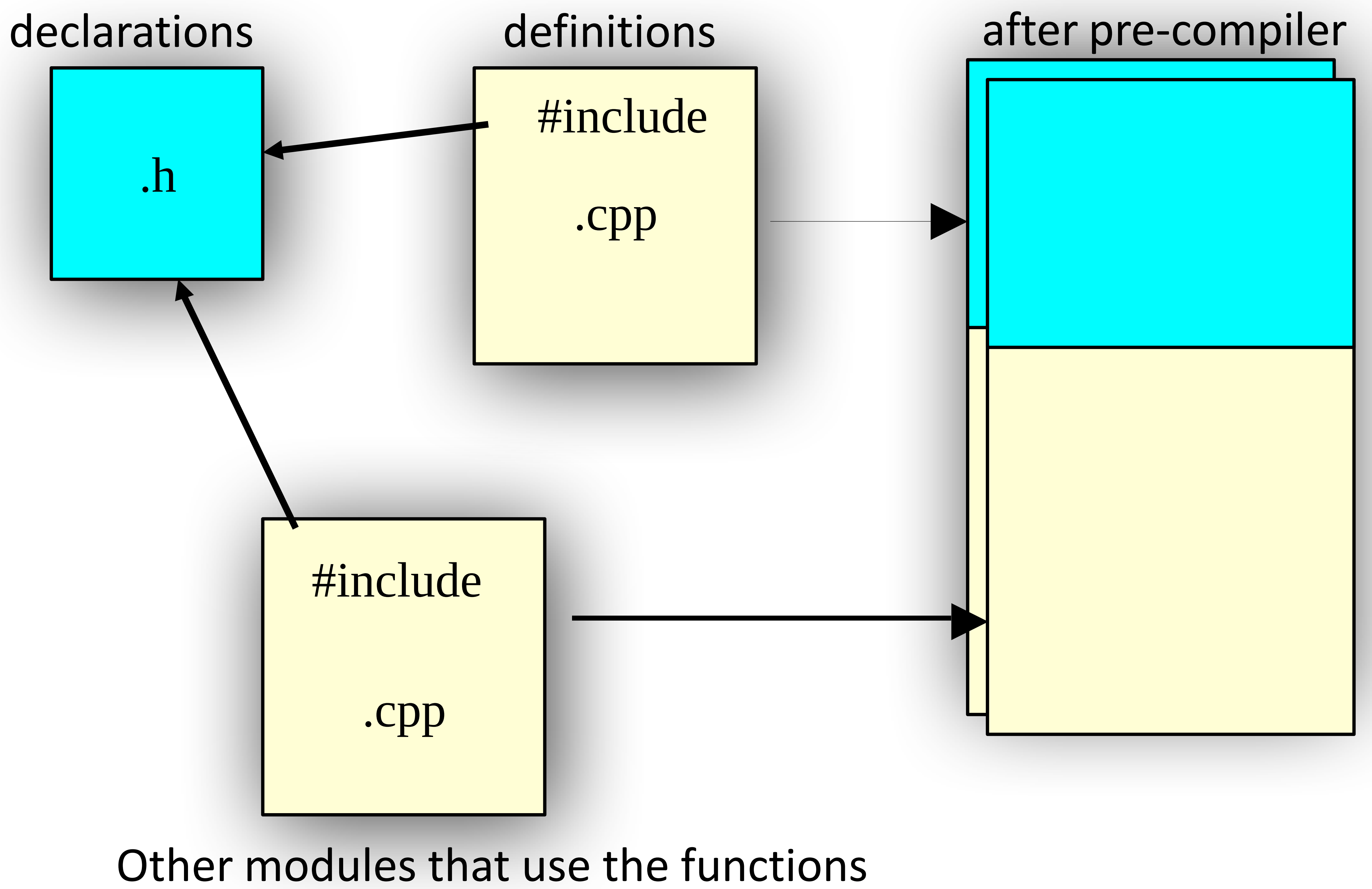
Structure of C++ program



Structure of C++ program



Structure of C++ program



Declarations vs. Definitions

- A .cpp file is a compile unit
- Only declarations are allowed to be in .h
  - extern variables
  - function prototypes
  - class/struct declaration

#include

#include

- `#include` is to insert the included file into the .cpp file at where the `#include` statement is.
  - `#include "xx.h"`: usually search in the current directory, implementation defined
  - `#include <xx.h>`: search in the specified directories

Standard header file structure

```
#ifndef HEADER_FLAG
#define HEADER_FLAG
// Type declaration here...
#endif // HEADER_FLAG
```

Tips for header

1. One class declaration per header file
2. Same name with .cpp file.
3. The contents of a header file is surrounded with
   `#ifndef` `#define` … `#endif`

The CMake utility